

Project Report

COMP2021 Object-Oriented Programming (Fall

2025)

Group 17

Members and contribution percentages:

UNG Sethy Pagna 23100448d: 40%

Law Shun Hin 24093082d: 32%

ERGIN Sadettin Uygur 25089266d: 28%

Table of Contents

1. Introduction.....	3
2. The Command Line Vector Graphics Software (CleviS).....	3
2.1. Design.....	3
2.2. Implementation of Requirements.....	4-8
3. Learning-to-Learn.....	8-9

1. Introduction

This report outlines the design and implementation of the Command Line Vector Graphics Software (CleviS), a project completed by Group XYZ for the course COMP2021 Object-Oriented Programming at PolyU.

2. The Command Line Vector Graphics Software (CleviS)

In this section, we describe first the overall design of the CleviS tool and then the implementation details of the requirements.

2.1. Design

The cleviS system is in modular architecture, divided into three main components to build a vector graphics tool. The core of the system is the CleviS class, the main application class that interacts with ShapeManager, CommandParser and CleviSLogger components. This design demonstrates a clean and cooperative design between the modules. CommandParser translating input into executive commands, CleviSLogger tracking users activities to meet the logging requirements, ShapeManager maintaining a clean and sustainable database for the shape collection.

The system is organized into three main packages:

- Model

Contains 'ShapeManager' for maintaining the shape organization, the abstract 'Shape' class hierarchy that enforces polymorphic behavior, and 'CleviSLogger' recording system activities and user commands.

- View

'CleviSGUI, DrawingPanel' provides the graphical and 'ConsoleView' for the interfaces for interactive window with users.

- Controller

Includes the main 'Clevis' class with embedded 'CommandParser' for command executions and coordinating between model and view components.

Our clevis system follows the MVC design pattern, where we enforced a clean separation between the main components between data management, GUI and the logic, creating a maintainable and extensible architecture.

2.2. Implementation of Requirements

For each (compulsory and bonus) requirement, describe 1) whether it is implemented and, when yes, 2) how you implemented the requirement as well as 3) how you handled various error conditions.

[REQ1] Logging Operations

- 1) Implementation Status: Fully implemented
- 2) Implementation Details: The ClevisLogger class handles dual-format logging to both HTML and text files. The operation will be stored in the logger, even if they failed. In HTML, the table rows are created with operation indices
- 3) Error Handling: 'Files.createDirectories(DIRECTORY_PATH)' ensures the directory is valid. The logger 'close()' method closes both the writers and completes the HTML structure, 'log()' ensures validity.

[REQ2] Drawing Rectangles

- 1) Implementation Status: Fully implemented
- 2) Implementation Details: The Rectangle class extends the abstract Shape class, storing position (x,y), width, and height. The 'rectangle n x y w h' command creates instances through CommandParser validation, with the shape added to ShapeManager for centralized management.

3) Error Handling: Validates exactly 5 parameters for 'rectangle n x y w h', checks for positive width/height values, and prevents duplicate and null shape names.

[REQ3] Drawing Line Segments

1) Implementation Status: Fully implemented

2) Implementation Details: The Line class implements the Shape interface with two endpoints (x1,y1) and (x2,y2). The 'line n x1 y1 x2 y2' command creates line segments that can be manipulated like other shapes.

3) Error Handling: Validates exactly 5 parameters, no identical endpoints and prevents duplicate and null shape names.

[REQ4] Drawing Circles

1) Implementation Status: Fully implemented

2) Implementation Details: The Circle class stores center coordinates (x,y) and radius r. The 'circle n x y r' command creates circles that support all shape operations including movement and bounding box calculation.

3) Error Handling: Validates exactly 4 parameters, positive radius and prevents duplicate and null shape names.

[REQ5] Drawing Squares

1) Implementation Status: Fully implemented

2) Implementation Details: The Square class extends the Shape hierarchy with position (x,y) and side length. The 'square n x y l' command creates squares that inherit all shape functionalities.

3) Error Handling: Validates parameter count, numeric values, checks for positive side length and prevents duplicate and null shape names.

[REQ6] Grouping Shapes

1) Implementation Status: Fully implemented

2) Implementation Details: The Group class implements the composite pattern, allowing multiple shapes to be treated as a single entity. The 'group n n1 n2 ...' command creates groups that support recursive operations. Stores encapsulated shapes in an ArrayList<shape>, only read is allowed via 'Collections.unmodifiableList()'.

3) Error Handling: Validates at least 3 parameters, checks existence of all member shapes, prevents empty groups and prevents duplicate and null group names.

[REQ7] Ungrouping Shapes

1) Implementation Status: Fully implemented

2) Implementation Details: Use 'instanceof Group' to check if the target actually is in a group before ungrouping. The 'ungroup n' command decomposes groups back into individual shapes, restoring them to the ShapeManager as independent entities.

3) Error Handling: Validates exactly 1 parameter, ensures the target exists and is actually a group, handles shape restoration conflicts.

[REQ8] Deleting Shapes

1) Implementation Status: Fully implemented

2) Implementation Details: The 'delete n' command removes shapes from ShapeManager by using 'shapesByName.remove(name)' to delete shapes from the LinkedHashMap. For groups, it recursively deletes all member shapes as required.

3) Error Handling: Validates exactly 1 parameter, and ensures the target shape exists before attempting deletion and checks for null shapes before operating..

[REQ9] Calculating Bounding Box

1) Implementation Status: Fully implemented

2) Implementation Details: Each shape implements getBoundingBox() returning a consistent decimal format.

3) Error Handling: Validates exactly 1 parameter, ensures shape existence and handles bounding box calculation errors for complex groups.

[REQ10] Moving Shapes

1) Implementation Status: Fully implemented

2) Implementation Details: The 'move n dx dy' command translates shapes by x and y. Groups move all member shapes recursively.

3) Error Handling: Validates exactly 3 parameters, ensures numeric dx/dy values, and confirms shape existence.

[REQ11] Finding Topmost Shape at Point

1) Implementation Status: Implemented with simplification

2) Implementation Details: Uses 'Collections.reverse(shapeList)' to perform reverse Z-Order search, from the last-added to the first-added shapes. The 'shapeAt x y' command checks shapes in reverse insertion order (top-to-bottom Z-order). Uses coversPoint() method for bounding box hit testing.

3) Error Handling: Validates exactly 2 parameters and numeric values, returns message when no shapes covers the point.

[REQ12] Checking Shape Intersection

1) Implementation Status: Implemented with bounding box approach

2) Implementation Details: The 'intersect n1 n2' command checks bounding box overlap between shapes using rectangle overlap logic (Axis-Aligned Bounding Box) collision detection. Returns "Yes" if bounding boxes share internal points. Parse the bounding box strings to double to extract coordinates by using 'Double.parseDouble()'.

3) Error Handling: Validates exactly 2 parameters and ensures both shapes exist, uses try-catch around 'Double.parseDouble()' to prevent anomalies.

[REQ13] Listing Shape Information

1) Implementation Status: Fully implemented

- 2) Implementation Details: The 'list n' command displays shape-specific information using 'getInfo()' method. Shows the parameters for shapes.
- 3) Error Handling: Validates exactly 1 parameter and ensures shape existence.

[REQ14] Listing All Shapes

- 1) Implementation Status: Fully implemented
- 2) Implementation Details: The 'listAll' command displays all shapes in insertion order (bottom-to-top Z-order) using each shape's getInfo() method.
- 3) Error Handling: Handles empty shape lists and maintains consistent output format.

[REQ15] Quit Command

- 1) Implementation Status: Fully implemented
- 2) Implementation Details: The 'quit' command terminates the application safely, closing and cleanup log files and freeing resources before exit.
- 3) Error Handling: Ensures proper cleanup even if errors occur during shutdown. Ensures HTML documents are closed properly.

[BON1] Graphical User Interface

- 1) Implementation Status: Fully implemented
- 2) Implementation Details: The GUI serves as the primary window on organizing different functions, it supports quick creation of shapes based on buttons and text command input for concise implementation, function with zoom with 'zoomFactor' between 0.1x and 5.0x, and dynamic updates after each operation.
- 3) Error Handling: Each shape drawing methods include try and catch to prevent invalid data, handling parser execution with wraps to prevent GUI crash, and ensures zoom scaling lies between the min and max zoom.

[BON2] Undo/Redo Functionality

- 1) Implementation Status: Fully implemented

- 2) Implementation Details: Undo and Redo Button to the main toolbar, then go back to the last operation you did, the button states are refreshed every command, displaying 'nothing to undo' when disabled.
- 3) Error Handling: The button are disabled when no operations have done, handling same exception as regular commands in the parser

3 Learning-to-Learn

Use this section to 1) reflect on your learning-to-learn experiences in the project and 2) describe your plan to improve your self-learning approaches.

Law Shun Hin:

1. Reflection on Learning Experiences

During the development of the Clevis project, my part of work is about Shape hierarchy and part of ShapeManager classes. Working on those provides me significant learning opportunities on applying object-oriented programming concepts, while building a project in a modular architecture makes it easy with maintenance and extension. Learning inheritance can reduce redundant codes, such as Circle extends Shape, where there are common behaviors that are shared by shapes, and specialize the geometric implementation of each type of shape.

This is my first time to cooperate with others in a coding group project, which gives me a brand new experience. I have learnt a lot from them, learning how to locate the file directories, codespace setups. Building a well structure of a coding project is crucial, which makes fixing code by adding or removing codes more flexible. Furthermore, revising the codes from others gained me a new perspective of coding style. For example, IF operation does not need brackets, it

can occupy for only one row. This approach reduces the redundancy and complexity of code, making coding more smooth and readable.

2. Plan to Improve Self-Learning

Structured pattern learning, I can apply the concept of modularity, breaking down a system into discrete, independent components, which will be beneficial in future programming projects.

Furthermore, it could also boost effectiveness on working or studying, breaking down the work into small tasks can make it more capable of handling. Clear structure can know what I am doing, prevent being overwhelmed by work and progress tracking.

UNG Sethy Pagna (James):

1. Reflection on Learning Experiences

My main role in this project was to implement core requirements (6, 7, and 11–15). I also supported my teammates by reintegrating the system and restructuring the overall design so everything worked as a coherent whole.

At the start, we did not fully understand the specification. We did an agile approach, which resulted in an oversized file combining almost all the classes. While working on my tasks and helping with integration, I realized how this affected scalability and consistency. After revisiting the project PDF and learning the MVC (Model-View-Controller) pattern, I helped reorganise the code and files into a clearer modular structure: the Controller (Clevi.java) handling logic and commands, the Model containing all shapes and supporting features like the logger and exceptions, and the View managing the GUI and console interface. This restructuring significantly improved readability, cohesion, and maintainability.

Through this experience, I learned that effective object-oriented design goes beyond fulfilling requirements. It depends on a well-structured and organized system. It taught me the importance of planning in maintaining a clear design.

2. Plan to Improve Self-Learning

In future projects, I will analyze specifications more carefully before implementation, design system architecture earlier using diagrams and requirement mapping, and participate more actively in design discussions. I will also regularly reflect on my contributions so I can continue improving both my technical skills and collaborative effectiveness.

Uyger Ergin:

1. Reflection on Learning Experiences

My primary contributions to this project focused on developing the graphical user interface and core infrastructure components. I implemented the ClevisGUI class, DrawingPanel for visual rendering, the ClevisLogger system for comprehensive logging, and modified the Clevis controller class to integrate these components effectively.

Initially, I approached the GUI implementation by creating basic Swing components, but quickly realized the importance of proper separation between visualization and business logic. Through iterative development, I learned to design the DrawingPanel to work cohesively with the shape models while maintaining the MVC architecture. The challenge of real-time updates between command execution and visual feedback taught me valuable lessons about observer patterns and event-driven programming.

Developing the ClevisLogger class provided insights into file I/O operations and the importance of persistent data management. I implemented both text and HTML logging formats to enhance debugging capabilities and user experience. Modifying the Clevis controller to coordinate between the model, view, and logging systems deepened my understanding of how controllers serve as the backbone of an application, managing data flow while maintaining loose coupling between components.

The most significant learning came from understanding how GUI components interact with underlying business logic. Implementing zoom functionality, bounding box visualization, and real-time command execution required careful consideration of thread safety and performance

optimization. I learned that effective GUI design isn't just about visual appeal but about creating intuitive interactions that enhance the user's workflow.

2. Plan to Improve Self-Learning

In future projects, I will deepen my understanding of UI/UX design principles to create more intuitive interfaces. I plan to explore advanced graphics programming techniques for better rendering performance and implement more sophisticated design patterns for complex user interactions. I will also focus on writing more comprehensive documentation for GUI components to facilitate team collaboration and maintenance. Additionally, I aim to strengthen my skills in automated testing for graphical interfaces to ensure reliability across different environments.